

航空票务数据库系统说明文件

杜诚俊 何韵琳 杨子卿 吴沁远

2026 年 7 月

目录

1 项目基本情况	4
2 系统需求分析	4
2.1 系统目标	4
2.2 用户角色	5
2.3 主要业务流程	5
2.3.1 旅客购票流程	5
2.3.2 退票流程	5
2.3.3 改签流程	6
2.3.4 管理员维护流程	6
3 数据库概念模型设计	6
3.1 核心实体	6
3.2 实体联系说明	7
3.3 航段设计说明	7
4 数据库逻辑设计	7
4.1 主要关系模式	7
4.2 主键与外键设计	8
4.3 完整性约束与业务约束	8
5 功能设计	9
5.1 旅客端功能	9
5.2 管理员端功能	9
6 模块划分	10
6.1 前端模块	10
6.2 后端模块	10

目录	2
7 系统实现说明	10
7.1 前后端通信	10
7.2 订单状态	11
7.3 库存处理	11
7.4 会员积分和 VIP	11
7.5 隐私保护	11
8 安装与运行说明	12
8.1 环境准备	12
8.2 创建数据库	12
8.3 导入 SQL 文件	12
8.4 配置后端数据库密码	13
8.5 启动后端	13
8.6 启动前端	13
9 使用说明	14
9.1 演示账号	14
9.2 旅客端推荐演示流程	14
9.3 退票演示流程	14
9.4 改签演示流程	15
9.5 管理员端推荐演示流程	15
10 测试说明	15
10.1 登录注册测试	15
10.2 航班查询测试	16
10.3 下单与支付测试	16
10.4 会员积分测试	16
10.5 退票测试	17
10.6 改签测试	17
10.7 管理员功能测试	18
11 常见问题	18
11.1 java 命令无法识别	18
11.2 数据库密码错误	18
11.3 数据库不存在	18
11.4 表不存在	19
11.5 前端页面能打开但登录失败	19
11.6 端口被占用	19

目录	3
12 总结	19

1 项目基本情况

本项目是数据库课程设计中的航空票务数据库系统。系统主要围绕旅客购票和管理员维护基础数据两个部分展开。旅客端实现注册、登录、查询航班、下单、支付、查看订单、退票、改签、餐食预订、会员积分和 VIP 状态展示等功能；管理员端实现系统概览、航班管理、航段管理、城市、机场、飞机、餐食等资源维护，以及用户和订单信息查看。

系统采用前后端分离架构。前端负责页面展示、用户输入和接口调用；后端负责业务逻辑处理、数据校验、事务控制和数据库访问；数据库负责保存用户、城市、机场、飞机、航班、航段、订单和餐食等业务数据。

项目主要分为以下几个部分：

- database：数据库建表脚本和初始化数据。
- backend：Spring Boot 后端项目，负责接口、业务逻辑和数据库访问。
- frontend：React 前端项目，负责旅客端和管理员端页面。
- docs：项目说明、接口文档和前后端对接文档等。

前端不直接连接数据库，而是统一向后端发送 HTTP 请求。后端收到请求后，根据业务逻辑访问 MySQL 数据库，并将结果以 JSON 格式返回给前端。

2 系统需求分析

2.1 系统目标

本系统的目标是实现一个能够完成航空票务基本业务流程的数据库应用系统。系统通过 MySQL 保存业务数据，通过 Spring Boot 提供后端接口，通过 React 前端展示页面，使用户能够完成航班查询、下单、支付、退票、改签等操作，也使管理员能够维护基础数据和查看系统运行情况。

系统主要目标如下：

1. 实现航空票务核心数据的集中存储和统一管理。
2. 支持旅客完成注册、登录、查询航班、下单、支付、退票、改签和查看订单。
3. 支持管理员查看系统概览，并维护航班、航段、城市、机场、飞机、餐食等基础数据。
4. 通过主键、外键、唯一约束和状态字段保证数据一致性。
5. 在购票过程中对航段库存进行扣减和恢复，避免明显的余票错误。
6. 对身份证号、密码哈希等敏感信息进行保护，不在前端页面直接展示。

2.2 用户角色

当前系统主要包含两类用户。

- 旅客：注册、登录、查询航班、创建订单、支付订单、查看订单、退票、改签、预订餐食、查看会员积分和 VIP 状态。
- 管理员：查看系统概览，维护城市、机场、飞机、航班、航段、餐食等基础数据，查看用户和订单列表。

系统登录后根据后端返回的用户类型进入不同页面。旅客进入旅客端，管理员进入管理员端。前端不让用户手动选择身份，而是以后端登录结果为准。

2.3 主要业务流程

2.3.1 旅客购票流程

旅客购票的主要流程如下：

1. 旅客登录系统。
2. 选择出发机场、到达机场和日期，查询可售航段。
3. 选择某一航段和舱位，填写乘机人姓名、身份证号和餐食选项。
4. 提交下单请求，系统生成待支付订单。
5. 订单生成后，后端扣减对应航段余票。
6. 旅客在规定时间内支付订单。
7. 支付成功后，订单状态变为 PAID，用户积分增加。
8. 如果积分达到 VIP 门槛，系统更新会员等级。

2.3.2 退票流程

退票流程如下：

1. 旅客在订单列表中选择一张 PAID 状态订单。
2. 点击退票按钮。
3. 后端判断订单状态是否允许退票。
4. 退票成功后，订单状态变为 REFUND SUCCESS。
5. 系统恢复对应航段库存，并回退相应积分。

2.3.3 改签流程

改签流程如下：

1. 旅客选择一张 PAID 状态订单。
2. 选择新的目标航班或航段。
3. 提交改签申请。
4. 后端创建新的改签订单，并通过 originalTicketId 记录与原订单的关系。
5. 如果需要补差价，旅客支付改签订单。
6. 改签完成后，旧票状态变为 CHANGE SUCCESS，新票状态变为 PAID。

2.3.4 管理员维护流程

管理员登录后可以进行基础数据维护。系统不进行破坏性删除。对于航班等对象，前端提供停用和恢复操作；对于不适合删除的对象，只提供查看或修改，避免破坏历史订单数据。

3 数据库概念模型设计

3.1 核心实体

根据当前系统 ER 图和业务实现，系统围绕 9 张核心表展开。主要实体如下：

- User：保存登录账号、用户姓名、用户类型、手机号、邮箱、积分、会员等级等信息。
- City：保存城市名称、城市代码和所属国家。
- Airport：保存机场代码、机场名称、所属城市和是否国际机场。
- Aircraft：保存飞机注册号、机型、制造商、座位数和飞机状态。
- Flight：保存航班号、日期、飞机、出发机场、到达机场和航班状态。
- FlightSegment：保存可售航段，包括起点站序、终点站序、起降时间、余票和价格。
- TicketSale：保存订单和机票记录，包括订单号、用户、航班、航段、舱位、状态、金额和时间。
- MealOption：保存可选择的餐食信息。
- MealReservation：保存订单与餐食之间的预订关系。

3.2 实体联系说明

系统中的主要联系如下：

1. 一个城市可以包含多个机场，一个机场属于一个城市。
2. 一架飞机可以执行多个航班，一个航班使用一架飞机。
3. 一个航班可以包含多个可售航段。
4. 一个旅客可以拥有多条订单记录。
5. 一条订单记录对应一个航班和一个航段。
6. 一条订单可以选择餐食，餐食预订通过 MealReservation 记录。
7. 改签订单通过 originalTicketId 指向原订单，从而形成订单之间的关联。

3.3 航段设计说明

本系统没有把用户购买的对象简单理解为整条航班，而是设计了 FlightSegment 作为实际可售单位。若航班路线为 A-B-C，则可以产生 A-B、B-C、A-C 等不同可售区间。这样可以更好地表示经停航班中的不同乘坐区间，也便于分别维护余票和价格。

4 数据库逻辑设计

4.1 主要关系模式

根据 ER 图，当前系统的主要关系模式如下。字段以项目实际数据库和后端实体为准，本文只列出核心字段。

- User: UserId, LoginAccount, UserName, IdNumberDigest, PasswordHash, UserType, PhoneNumber, Email, Points, MemberLevel, CreatedAt, UpdatedAt。
- City: CityId, CityName, CityCode, Country。
- Airport: AirportCode, AirportName, CityId, IsInternational。
- Aircraft: AircraftRegNo, AircraftType, Manufacturer, TotalFirstClassSeats, TotalEconomySeats, Status, Remark。
- Flight: FlightId, FlightNumber, FlightDate, AircraftRegNo, FlightStatus, DepartureAirportCode, ArrivalAirportCode, Remark。
- FlightSegment: SegmentId, FlightId, OriginStopNo, DestinationStopNo, OriginAirportCode, DestinationAirportCode, PlannedDepartureTime, PlannedArrivalTime, FirstClassRemainingSeats, EconomyRemainingSeats, FirstClassPrice, EconomyPrice, Remark。

- TicketSale: TicketId, OrderNo, UserId, FlightId, SegmentId, CabinClass, TicketStatus, PassengerName, PassengerIdNumberDigest, PriceAmount, PaymentAmount, OriginalTicketId, BookedAt, PaidAt, IssuedAt, ChangedAt, RefundedAt, ExpiredAt。
- MealOption: MealId, MealName, MealType, IsAvailable, Description。
- MealReservation: MealReservationId, TicketId, MealId, Quantity。

4.2 主键与外键设计

各表通过主键和外键保证数据之间的参照完整性。主要关系包括：

1. Airport 的 CityId 引用 City 的 CityId。
2. Flight 的 AircraftRegNo 引用 Aircraft 的 AircraftRegNo。
3. Flight 的 DepartureAirportCode 和 ArrivalAirportCode 引用 Airport 的 AirportCode。
4. FlightSegment 的 FlightId 引用 Flight 的 FlightId。
5. FlightSegment 的 OriginAirportCode 和 DestinationAirportCode 引用 Airport 的 AirportCode。
6. TicketSale 的 UserId 引用 User 的 UserId。
7. TicketSale 的 FlightId 引用 Flight 的 FlightId。
8. TicketSale 的 SegmentId 引用 FlightSegment 的 SegmentId。
9. TicketSale 的 OriginalTicketId 引用 TicketSale 的 TicketId，用于记录改签关系。
10. MealReservation 的 TicketId 引用 TicketSale 的 TicketId。
11. MealReservation 的 MealId 引用 MealOption 的 MealId。

4.3 完整性约束与业务约束

系统中主要约束包括：

1. 用户登录账号唯一。
2. 身份证号不保存明文，而是保存摘要。
3. 机场代码唯一。
4. 飞机注册号唯一。
5. 同一航班号和日期不能重复。

6. 同一航班中，起点站序必须小于终点站序。
7. 订单必须关联真实存在的用户、航班和航段。
8. 舱位余票不能小于 0。
9. 订单金额不能小于 0。
10. 订单状态只能在规定状态集合中变化。

5 功能设计

5.1 旅客端功能

旅客端主要功能如下：

- 注册：新旅客填写账号、密码、姓名、手机号、邮箱、身份证号后注册。
- 登录：使用账号和密码登录系统。
- 会员信息展示：展示积分、会员等级、VIP 门槛和折扣。
- 航班查询：按出发机场、到达机场和日期查询可售航段。
- 下单：选择航段、舱位、乘机人信息和餐食后创建订单。
- 支付：对待支付订单进行支付，支付成功后更新订单状态和积分。
- 我的订单：查看当前用户的全部订单。
- 退票：对已支付订单进行退票处理。
- 改签：对已支付订单申请改签，并生成新的改签订单。
- 注销账号：注销当前旅客账号，但不物理删除历史订单。

5.2 管理员端功能

管理员端主要功能如下：

- 系统概览：展示用户、航班、航段、订单、餐食等数量统计。
- 航班管理：新增、修改、停用和恢复航班。
- 航段管理：新增和修改可售航段，维护余票、价格和起降时间。
- 资源管理：维护城市、机场、飞机和餐食等基础数据。
- 用户与订单查看：查看用户列表和订单列表，但不展示敏感字段。
- 过期订单处理：手动触发过期待支付订单扫描，释放库存。

6 模块划分

6.1 前端模块

前端采用 React 和 TypeScript 实现，主要模块包括：

- App：前端根组件，控制登录状态、旅客端和管理员端页面切换。
- AuthScreen：登录和注册页面。
- PassengerWorkspace：旅客端页面，包含会员信息、航班查询、下单、订单、退票、改签等功能。
- AdminWorkspace：管理员端页面，包含概览、航班、航段、资源、用户和订单模块。
- api/client：底层 HTTP 请求封装，统一处理 GET、POST 和后端响应格式。
- api/airticket：业务接口封装，供页面组件调用。
- types：TypeScript 类型定义。
- styles.css：页面样式。

6.2 后端模块

后端采用 Spring Boot 实现，主要包括：

- Controller：接收前端请求，返回统一 JSON 响应。
- DTO：定义前后端传输的数据结构。
- Service：实现注册、登录、购票、支付、退票、改签、过期订单处理等业务逻辑。
- Repository：负责数据库访问。
- Entity：与数据库表对应的实体类。
- Scheduler：定时扫描过期待支付订单。
- Common：统一响应格式和异常处理。

7 系统实现说明

7.1 前后端通信

前端通过 fetch 请求后端接口。后端接口统一前缀为：

```
http://localhost:8080/api
```

查询类接口使用 GET，例如航班查询、订单查询、资源列表查询；状态变更类接口使用 POST，例如登录、注册、下单、支付、退票、改签、停用和恢复等。

后端统一返回格式为：

```
{
  "code": 0,
  "message": "success",
  "data": {},
  "timestamp": "2026-06-10T12:00:00"
}
```

前端不能只判断 HTTP 状态码，还需要判断返回体中的 code。若 code 不为 0，则显示后端返回的 message。

7.2 订单状态

当前系统主要使用以下订单状态：

- PENDING PAYMENT：待支付。
- PAID：已支付。
- EXPIRED：已过期。
- REFUND SUCCESS：退票成功。
- CHANGE SUCCESS：改签成功。

前端页面可以显示中文解释，但不自行编造新的状态名。

7.3 库存处理

下单成功后，后端会先锁定对应航段库存，使相应舱位余票减少。支付成功后订单正式变为已支付。若订单超过支付时间仍未支付，后端会将订单变为过期并释放库存。退票成功后，也会恢复对应航段库存。

7.4 会员积分和 VIP

旅客支付成功后增加积分。当积分达到 VIP 门槛时，会员等级变为 VIP。VIP 用户再次购票时，实付金额按照折扣计算。前端可以展示价格预览，但最终支付金额以后端返回结果为准。

7.5 隐私保护

身份证号由用户输入后传给后端，后端保存摘要信息。前端和管理员页面不展示身份证明文，也不展示 passwordHash、idNumberDigest 等敏感字段。

8 安装与运行说明

8.1 环境准备

运行项目之前，需要安装：

- MySQL
- Java 17
- Maven
- Node.js
- npm

可以用以下命令检查：

```
java -version
mvn -v
node -v
npm -v
```

8.2 创建数据库

登录 MySQL：

```
mysql -u root -p
```

创建数据库：

```
CREATE DATABASE IF NOT EXISTS airticket
DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
EXIT;
```

8.3 导入 SQL 文件

进入项目根目录：

```
cd "<项目根目录>"
```

导入建表脚本和演示数据。以下文件名仅按常见命名书写，实际提交时应改成仓库中真实 SQL 文件名：

```
mysql -u root -p --default-character-set=utf8mb4 airticket < ".\database\schema.sql"
```

```
mysql -u root -p --default-character-set=utf8mb4 airticket < ".\database\seed_data.sql"
```

如果项目中的 SQL 文件不是这两个名字，应以 database 目录下实际文件为准。

8.4 配置后端数据库密码

打开后端配置文件：

```
backend/src/main/resources/application.properties
```

检查数据库连接配置：

```
spring.datasource.url=jdbc:mysql://localhost:3306/airticket
spring.datasource.username=root
spring.datasource.password=<MySQL密码>
```

其中 spring.datasource.password 应改成本机 MySQL 的实际密码。

8.5 启动后端

进入后端目录：

```
cd "<项目根目录>\backend"
```

启动后端：

```
mvn spring-boot:run
```

后端启动后不要关闭该窗口。可以在浏览器中访问：

```
http://localhost:8080/api/admin/airport/list
```

如果返回 JSON 数据，说明后端和数据库连接正常。

8.6 启动前端

打开另一个 PowerShell 窗口，进入前端目录：

```
cd "<项目根目录>\frontend"
```

安装依赖：

```
npm install
```

启动前端：

```
npm run dev
```

浏览器访问：

```
http://localhost:5173
```

前端和后端需要同时保持运行。

9 使用说明

9.1 演示账号

系统准备了以下演示账号：

- 管理员：admin / admin123。用于管理员端演示。
- 旅客 A：passengerA / pass123。初始积分为 900，用于演示升级 VIP。
- 旅客 B：passengerB / pass123。用于普通旅客流程演示。

9.2 旅客端推荐演示流程

1. 使用 passengerA 登录。
2. 查看会员信息，确认积分为 900，等级为 NORMAL。
3. 查询 PEK 到 SHA，日期选择 2026-07-01。
4. 选择一个可售航段，填写乘机人信息并创建订单。
5. 查看订单状态 PENDING PAYMENT。
6. 点击支付，查看订单状态变为 PAID。
7. 查看积分变为 1000，会员等级变为 VIP。
8. 再次创建订单，查看 VIP 折扣是否生效。

9.3 退票演示流程

1. 在我的订单中选择一张 PAID 状态订单。
2. 点击退票。
3. 查看订单状态是否变为 REFUND SUCCESS。
4. 重新查询航班，观察对应航段库存是否恢复。

9.4 改签演示流程

1. 在我的订单中选择一张 PAID 状态订单。
2. 点击改签。
3. 选择新的目标航段。
4. 提交改签申请。
5. 若产生待支付改签单，则点击支付。
6. 查看旧票状态是否变为 CHANGE SUCCESS，新票状态是否为 PAID。

9.5 管理员端推荐演示流程

1. 使用 admin 登录。
2. 查看系统概览。
3. 查看航班列表。
4. 停用某个航班，观察状态变为 DISABLED。
5. 恢复该航班，观察状态变为 NORMAL。
6. 查看航段列表，说明航段是实际售票单位。
7. 查看订单列表，说明订单状态变化。
8. 手动触发过期订单扫描。

10 测试说明

10.1 登录注册测试

测试内容：

- 管理员账号能否登录。
- 旅客账号能否登录。
- 新旅客能否注册。
- 错误密码是否会提示失败。
- 手机号、邮箱、身份证格式错误时是否会提示。

预期结果：

- 合法账号能够进入对应页面。

- 错误输入不会写入异常数据。
- 后端返回的错误信息能在前端显示。

10.2 航班查询测试

测试内容：

- 按出发机场、到达机场和日期查询航班。
- 查询无结果路线。
- 查询经停航班的不同航段。

预期结果：

- 系统返回符合条件的可售航段。
- 页面正确显示航班号、时间、价格和余票。
- 无结果时页面显示空状态提示。

10.3 下单与支付测试

测试内容：

- 选择可售航段创建订单。
- 支付待支付订单。
- 观察订单状态变化。
- 观察库存变化。

预期结果：

- 创建订单后状态为 PENDING PAYMENT。
- 支付后状态变为 PAID。
- 下单后库存减少，退票或过期后库存恢复。

10.4 会员积分测试

测试内容：

- 使用 passengerA 登录。
- 支付第一张票。
- 查看积分和会员等级变化。

- 再次下单查看 VIP 折扣。

预期结果：

- passengerA 初始积分为 900。
- 支付后积分达到 1000。
- 会员等级变为 VIP。
- 后续购票时实付金额体现折扣。

10.5 退票测试

测试内容：

- 对 PAID 订单进行退票。
- 对非 PAID 订单尝试退票。
- 查看退票后的库存和积分变化。

预期结果：

- PAID 订单可以退票。
- 退票后订单状态变为 REFUND SUCCESS。
- 库存恢复，积分回退。
- 非 PAID 订单不能退票。

10.6 改签测试

测试内容：

- 对 PAID 订单申请改签。
- 选择新航段。
- 支付改签订单。
- 查看新旧订单状态。

预期结果：

- 旧票状态变为 CHANGE SUCCESS。
- 新票状态为 PAID。
- 新票 originalTicketId 指向旧票。

10.7 管理员功能测试

测试内容：

- 查看系统概览。
- 查看航班和航段列表。
- 停用和恢复航班。
- 查看用户和订单列表。
- 触发过期订单扫描。

预期结果：

- 管理员能够正常查看基础数据。
- 航班停用后状态变为 DISABLED。
- 航班恢复后状态变为 NORMAL。
- 管理员页面不展示密码哈希和身份证摘要等敏感字段。

11 常见问题

11.1 java 命令无法识别

如果 PowerShell 提示无法识别 java，说明 Java 没有安装或环境变量没有配置。需要安装 Java 17，并重新打开 PowerShell。

11.2 数据库密码错误

如果后端出现：

```
Access denied for user 'root'@'localhost'
```

说明 application.properties 中的 MySQL 密码和本机密码不一致，需要修改：

```
spring.datasource.password=<MySQL密码>
```

11.3 数据库不存在

如果后端出现：

```
Unknown database 'airticket'
```

说明还没有创建数据库，需要先执行：

```
CREATE DATABASE IF NOT EXISTS airticket
DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

11.4 表不存在

如果后端提示某张表不存在，说明数据库已经创建，但还没有导入建表脚本。需要重新导入 database 目录下的 SQL 文件。

11.5 前端页面能打开但登录失败

这种情况通常说明前端已经启动，但后端或数据库没有正常运行。应先访问：

```
http://localhost:8080/api/admin/airport/list
```

如果该地址不能返回正常 JSON，应优先检查后端启动情况和数据库连接情况。

11.6 端口被占用

如果 8080 或 5173 端口被占用，可以关闭原来的后端或前端窗口，然后重新启动项目。

12 总结

本项目实现了航空票务数据库系统的基本业务闭环。旅客端可以完成从查询航班到下单、支付、退票、改签和查看会员状态的流程；管理员端可以完成基础数据维护和订单查看。数据库设计以 ER 图为基础，通过主键、外键、唯一约束和状态字段维护数据之间的关系。

在实现过程中，系统重点处理了可售航段、订单状态、库存变化、会员积分和隐私字段展示等问题。由于项目时间有限，当前系统更侧重于课程设计所要求的数据库设计、业务流程完整性和系统演示稳定性，后续可以继续扩展更复杂的支付接口、航班实时动态、更多统计分析等功能。